

技术栈面试速记（7 个核心）

1. Java Spring Boot 3

用来干嘛：电商核心业务后端，8 张表的商品/购物车/下单/支付全靠它。四层架构（Controller → Service → Repository → Entity），Controller 层收请求鉴权后调 Service，Service 用 @Transactional 包事务，Repository 用 JPA 派生查询，写操作前加悲观锁防并发。

面试 Q1：为什么用 Spring Boot 不用自己搭 Servlet？ 答：自动配置省掉所有 XML，内嵌 Tomcat 一条命令启动。我的后端从建表到能跑起来没写一行 XML。

面试 Q2：@Transactional 加在哪？事务里做了啥？ 答：加在 OrderServiceImpl 和 OrderPaymentServiceImpl 类上。一个事务里同时更新订单状态 + 循环恢复所有商品的库存和销量——必须包在一起，不然支付成功了库存没回滚就炸了。

2. Python FastAPI

用来干嘛：Agent 服务主框架，暴露 /chat（非流式）、/chat/stream（SSE 流式）、/confirm（用户二次确认写操作）三个核心接口。原生 async/await，大模型调用是 IO 密集型用协程并发跑多个用户对话。

面试 Q1：为什么用 FastAPI 不用 Flask？ 答：FastAPI 有三件事对你这个场景至关重要：① 原生 async，大模型调用几百 ms 不阻塞；② 自带 Pydantic 校验空消息和超长 Prompt；③ SSE 流式推送一行代码搞定，不用自己写 WebSocket 心跳保活。

面试 Q2：SSE 流式推送和 WebSocket 区别？ 答：SSE 是 HTTP 连接上单向推消息（服务器→客户端），自动断线重连、nginx 反代只要去缓存就能用。WebSocket 是双向全双工，还要处理 TCP 升级和心跳。我的 Agent 场景只有服务器推工具执行轨迹，用 SSE 更轻。

3. Function Calling（工具调用）

用来干嘛：让大模型能调我写的 13 个电商工具（查商品、加购物车、取消订单等）。我把每个工具的 JSON Schema 喂给模型，模型返回 function_call，我解析参数、走三层防护（Schema 校验 + 业务意图守卫 + 5 轮上限）、执行工具、把结果塞回模型上下文继续下一轮。

面试 Q1：Function Calling 怎么工作的？ 答：三步走。① 给模型看一份函数说明书（JSON Schema）；② 模型判断要调工具时返回 function_call 名字和 arguments JSON；③ 我在代码里 json.loads 解析、校验、执行、把结果 function_call_output 塞回去，模型再决定是继续调工具还是给自然语言回答。

面试 Q2：模型乱传参数怎么办？ 答：两层防。第一层 JSON Schema strict=True + additionalProperties=False 卡死格式，多传少传直接拒；第二层执行前我写了 _guard_tool_call 业务意图守卫——用户原话没提「取消」两字，模型敢调 cancel_order 直接拦下来。

4. LangGraph

用来干嘛：Human-in-the-Loop 审批 workflow。写操作（取消订单、清空购物车、创建订单、支付）不会直接落库，而是走 LangGraph 的 interrupt() 挂起图执行，用户点确认后用 Command(resume=approved) 恢复图跑完。图状态用 Redis Checkpointer 持久化，服务重启不丢挂起的确认。

面试 Q1：LangGraph 和 LangChain 什么关系？ 答：LangChain 是组件库（Prompt/Tool/Memory/Chain），LangGraph 是编排引擎。LangChain 适合线性 pipeline，LangGraph 能做循环、分支、状态持久化、人类介入——我的 Agent 核心是「模型→调工具→模型」的循环 + 写操作的人类审批中断，用 LangGraph 做图编排比自己手写 while 循环清晰。

面试 Q2：interrupt() 机制是什么？和普通 await 区别？ 答：interrupt() 是状态快照挂起——它把当前图的节点位置、ApprovalState 序列化存到 Redis（Checkpointer），让出执行权。下次用同一个 thread_id + Command(resume=xxx) 恢复时，从上次 interrupt 的位置继续执行，resume 参数就是 interrupt 的返回值。普通 await 只是让出协程，状态在内存里，进程重启就丢了。

5. MCP (Model Context Protocol)

用来干嘛：把 Agent 的电商能力暴露成通用工具。任何支持 MCP 的 AI 客户端（Trae、Cursor、VS Code AI）都能直接调用 search_products、get_product_detail、get_my_orders 这 5 个工具，不用在每个客户端里重新写一遍函数调用协议。

面试 Q1：MCP 和 Function Calling 什么关系？ 答：Function Calling 是单个大模型在一次对话里调工具的协议（模型和应用之间）。MCP 是跨工具/跨应用/跨模型的统一协议——它定义了 tools/resources/prompts 三类能力，走 stdio 或 SSE 传输，任何 MCP 客户端 + 任何支持 MCP 的模型都能用同一套工具。类比：Function Calling ≈ HTTP 的一个具体 endpoint，MCP ≈ 整个 REST API 规范。

面试 Q2：MCP 为什么重要？ 答：不用 MCP，我每次想在新 AI 客户端里接入电商能力，都要重新写一遍「怎么调 search_products、参数格式是什么」——每家客户端的 Function Calling 格式不一样。用 MCP 后我只写一份工具定义，Trae、Cursor、以后任何 AI IDE 都能复用。

6. MySQL 8.0 + Redis

用来干嘛：MySQL 存 8 张电商表（用户、商品、购物车、订单等），全部 InnoDB 事务安全 + utf8mb4 支持中文/emoji。Redis 存四件事：① Agent 会话记忆（累积到 10 条消息触发规则摘要压缩）；② 会话状态（上一个订单/商品 ID，用来解「它、那个」这类代词）；③ 一次性确认令牌（写操作的二次确认，10 分钟 TTL + Lua CAS 脚本防重复消费）；④ LangGraph 审批中断快照。

面试 Q1：为什么会话记忆要压缩？压缩后还能恢复吗？ 答：因为大模型 context window 有限 + token 成本高。触发阈值是 10 条消息，取最近 6 条留原文，旧消息用正则抽取「商品 ID / 订单 ID / 预算 / 操作关键词」拼成一条历史摘要塞最前面。**不能精确恢复原文**，但关键业务实体都保留了——权衡是「宁可牺牲精度换能跑下去」。

面试 Q2：悲观锁在哪用的？为什么用悲观锁不用乐观锁？ 答：OrderRepository 的 findByIdForUpdate 方法上标了 @Lock(PESSIMISTIC_WRITE)，SQL 就是 SELECT ... FOR UPDATE。支付和取消订单前先锁这张订单行。用悲观锁因为**写冲突概率高**——同一张待支付订单可能同时被用户点取消和支付。乐观锁（version 字段）在高并发下会大量 retry 甚至丢单，悲观锁直接锁住行直到事务结束。

7. Docker + Docker Compose

用来干嘛：一键部署。docker-compose.yml 编排 mysql:8.0 → backend:8080（Spring Boot）→ agent:8000（FastAPI）三个服务，依赖启动顺序用 service_healthy 保证，MySQL 启动时自动跑 schema.sql + 初始化数据 + Agent 知识库 SQL。

面试 Q1：depends_on 的 service_started 和 service_healthy 区别？ 答：service_started 只要进程启动就算（不管能不能连），service_healthy 要等健康检查（mysqladmin ping 或 curl /health）通过才算。我的 MySQL 用了 service_healthy 因为刚启动时连不上，必须等 ready 了 Spring Boot 才能连。

面试 Q2：Redis 挂了会怎么样？系统能降级吗？ 答：能。Redis 是可选项不配也行——会话记忆用内存版 ConversationMemoryStore，会话状态用内存版 ConversationStateStore，确认令牌用内存版 ConfirmationStore，LangGraph Checkpointer 自动回退 InMemorySaver。功能全有但服务重启会丢会话——典型的降级开关。

技术栈 × 你项目里干什么（一眼扫完）

技术栈	你项目里干嘛的
Spring Boot 3	Java 电商后端，8 表 CRUD + JWT 鉴权 + 悲观锁防并发
FastAPI	Python Agent 框架，async 协程 + SSE 流式推送
Function Calling	让 LLM 调 13 个电商工具 + 三层防护防越权
LangGraph	写操作二次确认审批流 + interrupt/resume HITL
MCP	电商工具暴露给 Trae/Cursor 等 AI 客户端复用
MySQL 8	8 张 InnoDB 电商表，Agent 可直连或走 Java REST
Redis	会话记忆 + 会话状态 + 确认令牌 + LangGraph Checkpointer
Docker Compose	一键拉 MySQL/Backend/Agent 三容器